Pulling Google News Results with SearchApi.io

Into Azure Data Factory via a REST Linked Service, Landing Dated JSON Files in Blob Storage

---

This document follows the same pattern as the Walmart Search walkthrough, applied to the **Google News** engine on SearchApi.io. The key difference in this run: the Blob Storage sink uses dynamic expressions to build a dated folder path and a timestamped file name, so every pipeline run lands in its own uniquely named JSON file instead of overwriting the last one.

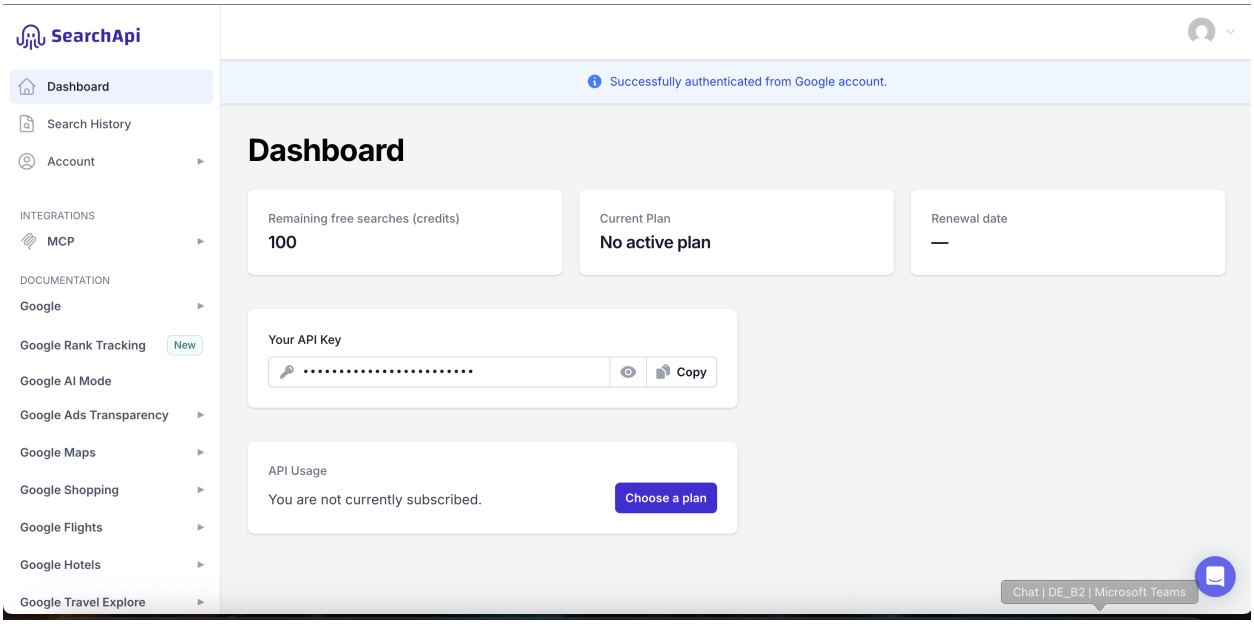
Tools involved: SearchApi.io | Azure Data Factory | Azure Blob Storage | a JSON viewer (jsonviewer.stack.hu)

Step 1: Find the Google News endpoint on SearchApi.io

In the SearchApi.io documentation, the **Google News** engine is selected. It exposes an automated news-aggregation endpoint that collects headlines from many sources and ranks them by relevance, recency, and location.

```
GET /api/v1/search?engine=google_news
```

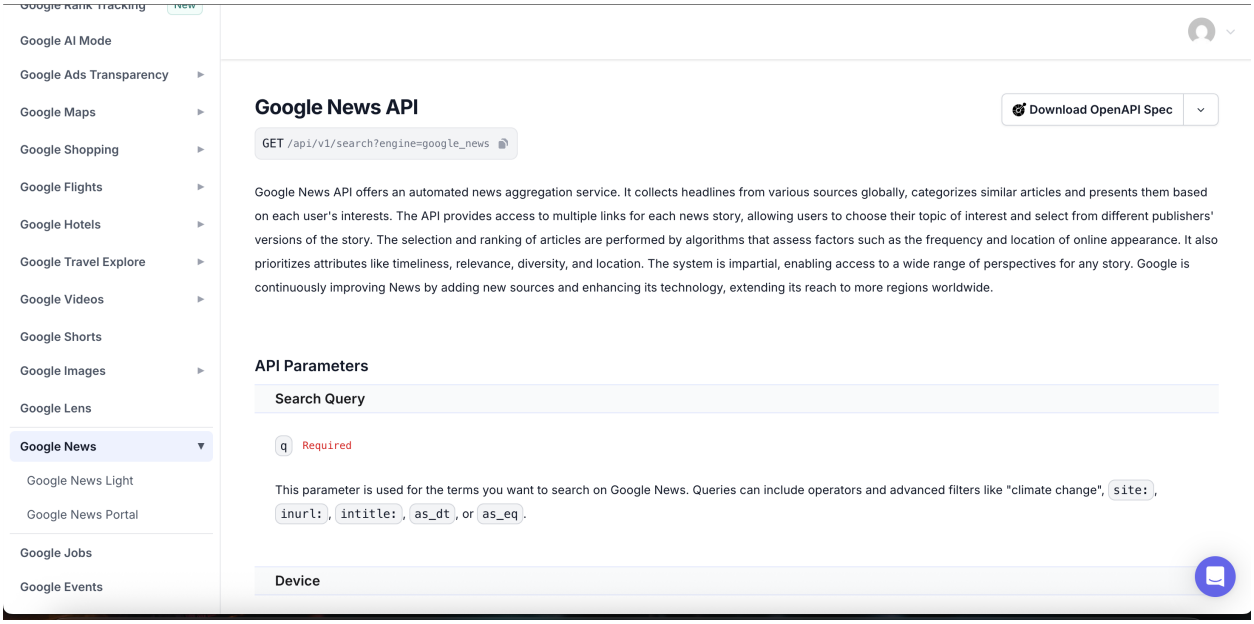
The only required parameter is **q**, the search query, which supports the same operators as a normal Google search (quoted phrases, **site:**, **inurl:**, **intitle:**, date filters, and so on).



SearchApi.io documentation page for the Google News API.

Step 2: Confirm the SearchApi account and grab the API key

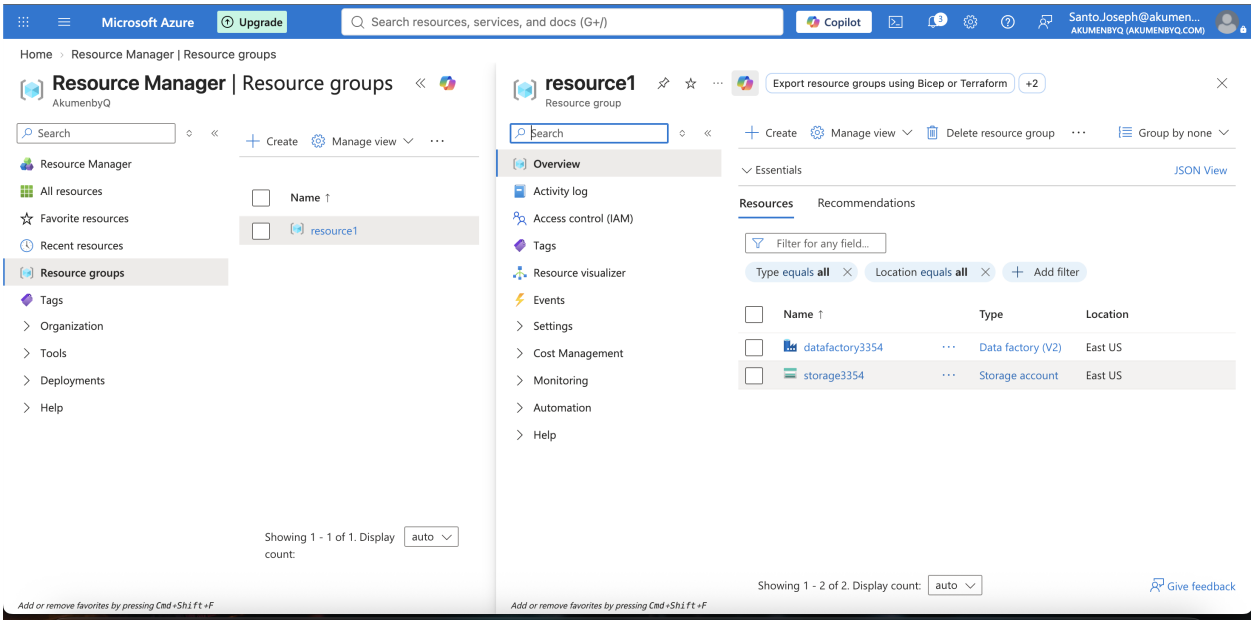
The SearchApi Dashboard confirms the account is signed in (authenticated via Google) and shows 100 remaining free-search credits, with no paid plan active yet. The **API Key** field on this page is what authenticates every request.



SearchApi Dashboard: authenticated session, 100 free credits, API key field.

Step 3: Provision the Azure resources

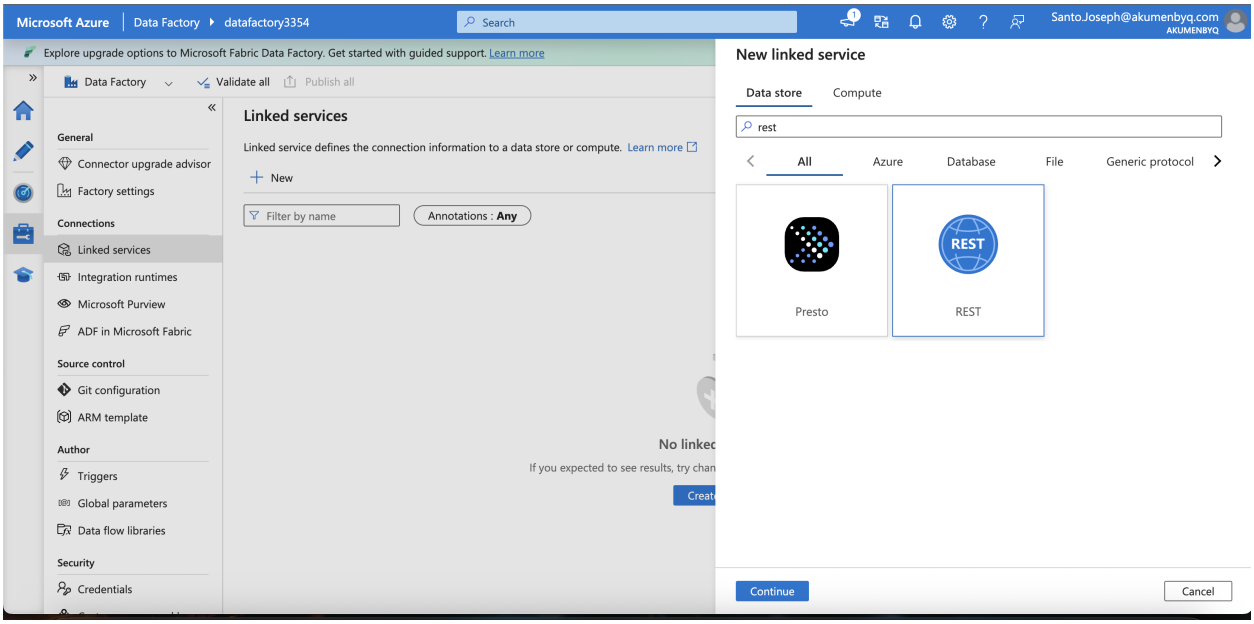
In the Azure Portal, a resource group (**resource1**) already contains the two resources this workflow needs: a **Data Factory** instance (**datafactory3354**) and a **Storage account** (**storage3354**) that will hold the JSON output.



Resource group with the Data Factory and Storage account already provisioned.

Step 4: Create a REST linked service in Azure Data Factory

Back in Data Factory (Manage → Linked services → New), searching "rest" surfaces the generic **REST** connector again — the same building block used for the Walmart Search integration, just pointed at a different SearchApi engine this time.



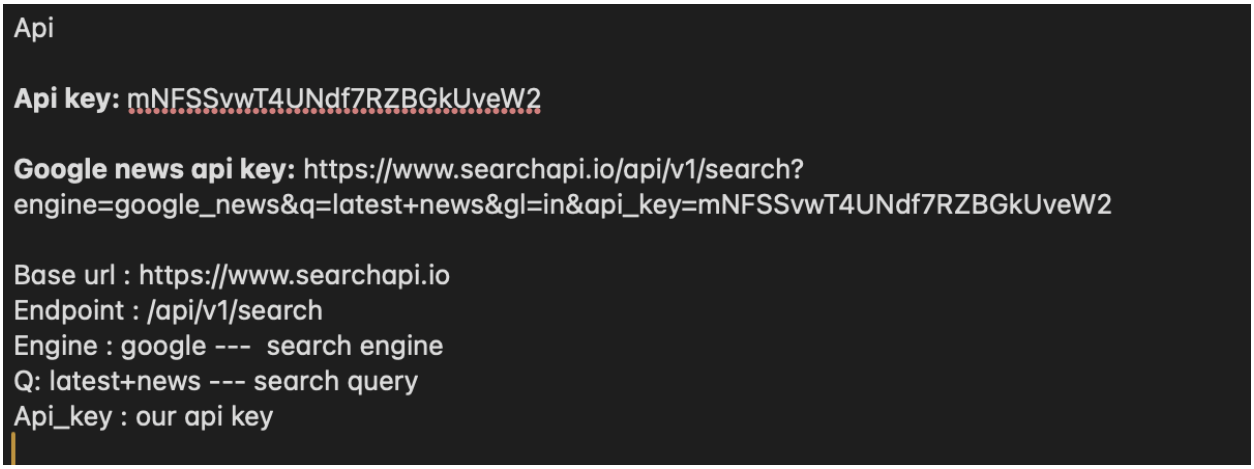
Choosing the REST connector for the new linked service.

Step 5: Assemble the full request URL

As before, the endpoint shape and the API key are combined into one working URL, this time querying "latest news" restricted to India (**gl=in**):

```
https://www.searchapi.io/api/v1/search?engine=google_news&q=latest+news&gl=in&api_key=[REDACTED]
```

(The API key has been redacted here for safety.)

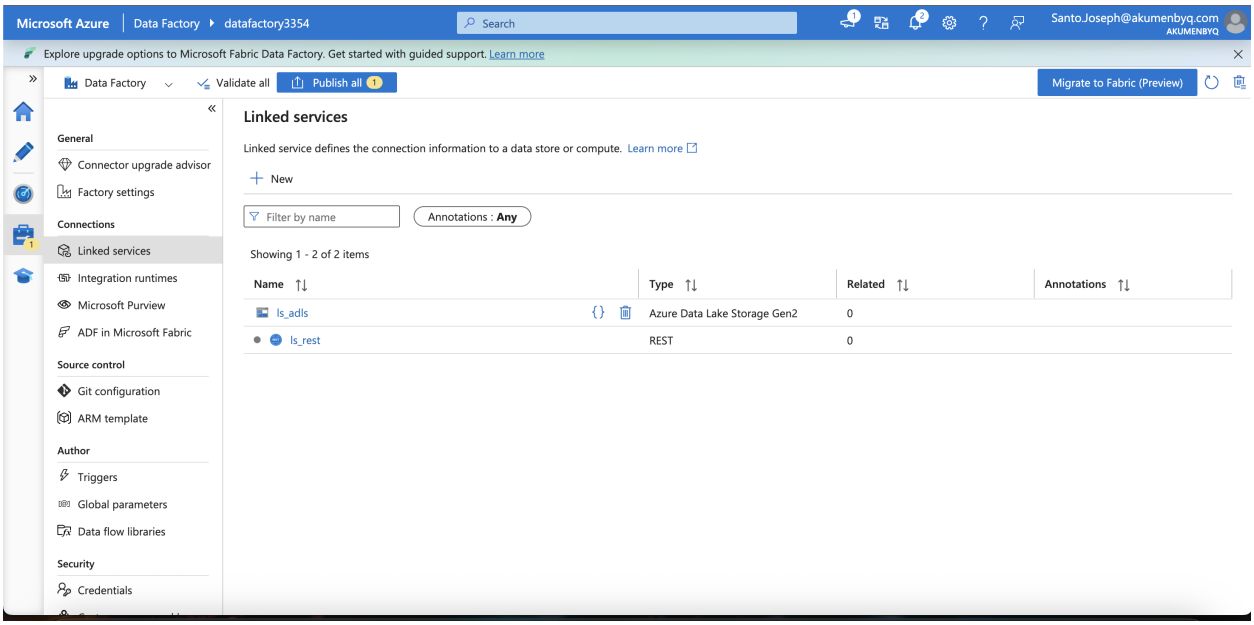


Notes breaking the URL down into base URL, endpoint, engine, q, and api_key.

Just like the Walmart walkthrough, this key should never be pasted into shared or public locations — anyone holding it can spend the account's search credits.

Step 6: Wire up both linked services

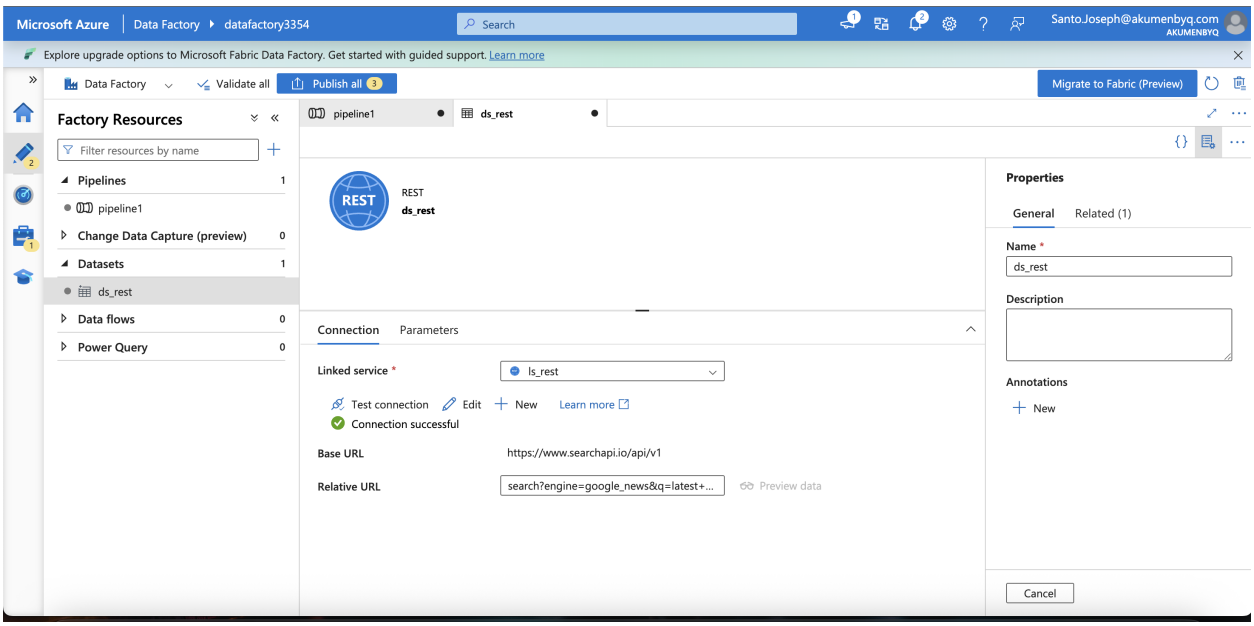
Two linked services now exist side by side in Data Factory: **ls_rest** (the REST connector pointed at SearchApi) and **ls_adls** (Azure Data Lake Storage Gen2 / Blob Storage, the eventual destination for the copied JSON).



Linked services list: ls_adls (storage) and ls_rest (REST/SearchApi).

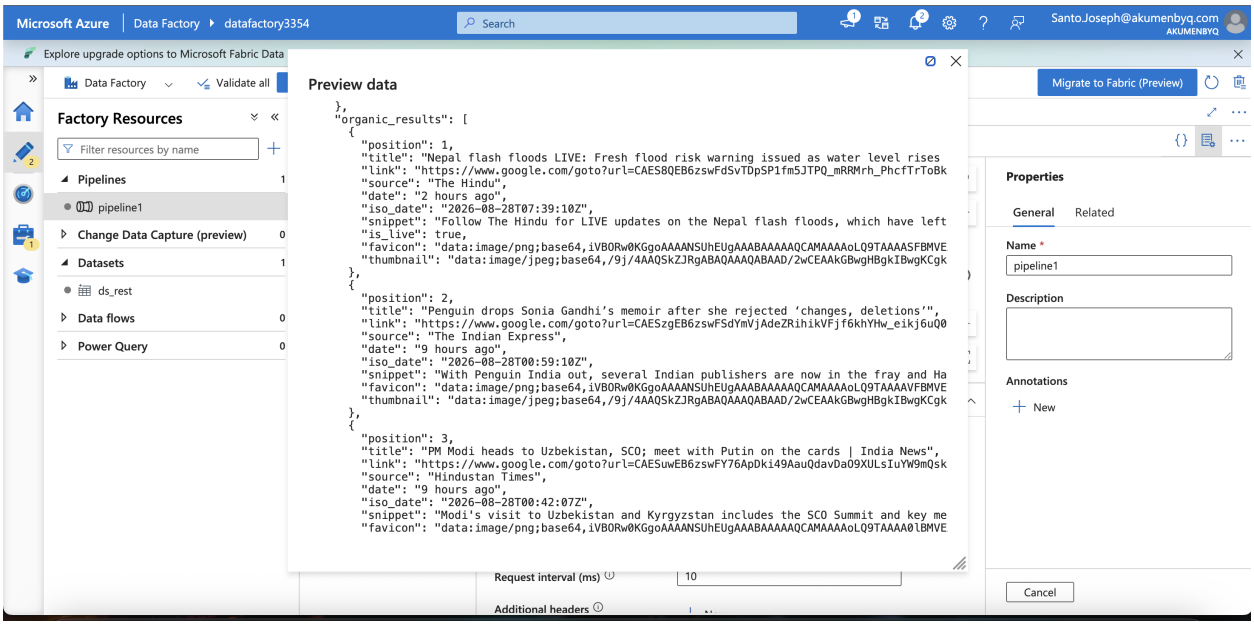
Step 7: Create the source dataset (ds_rest) and test it

A dataset named **ds_rest** sits on top of **ls_rest**. Its **Base URL** is **https://www.searchapi.io/api/v1** and its **Relative URL** holds the query string (**search?engine=google_news&q=latest+...**). Clicking **Test connection** confirms the setup reaches SearchApi successfully.



ds_rest dataset: Base URL, Relative URL, and a successful test connection.

The **Preview data** option on this dataset sends a live request and displays the JSON payload — here showing several **organic_results** entries (news headlines, sources, timestamps, snippets, and favicons) so the mapping can be sanity-checked before building the pipeline.

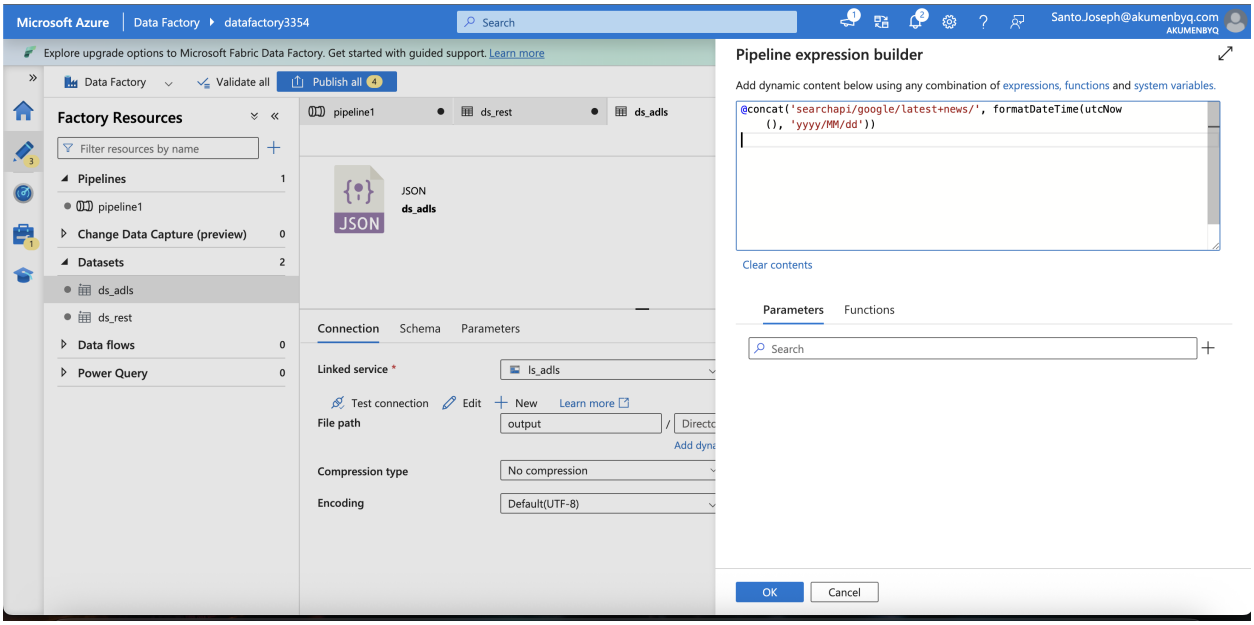


Live preview of the Google News JSON response.

Step 8: Build a dynamic destination path for the sink

Rather than writing every run to the same fixed blob, the sink dataset **ds_adls** (on **ls_adls**, inside the **output** container) uses the Data Factory pipeline expression builder to generate a **dated folder path** dynamically:

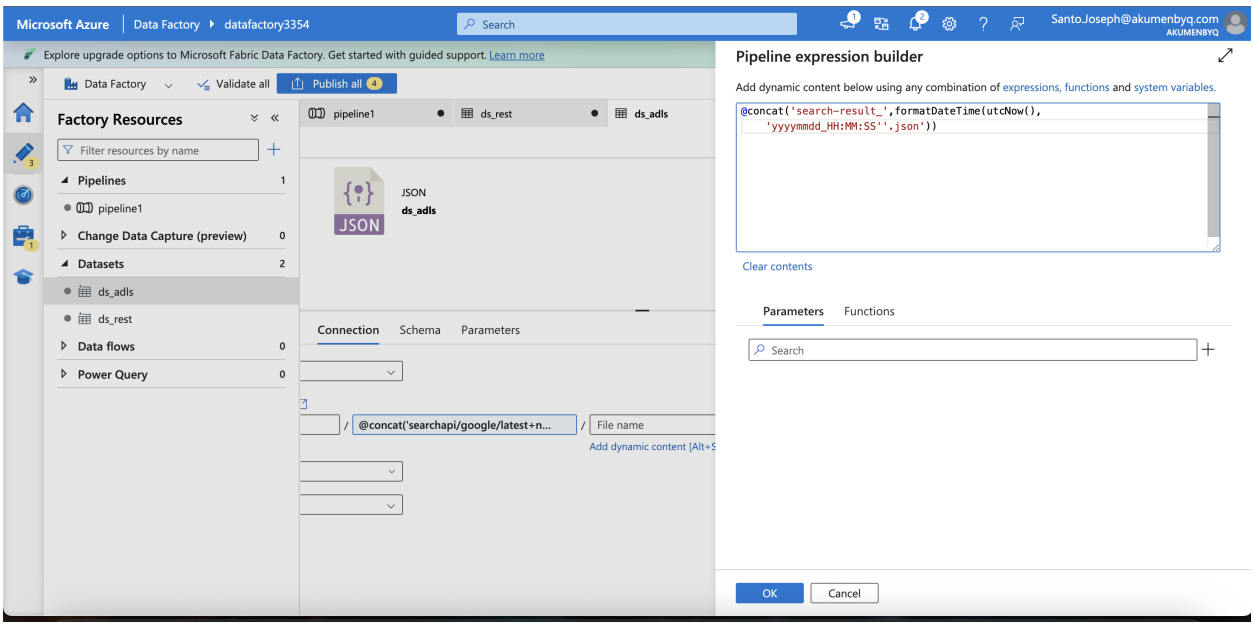
```
@concat('searchapi/google/latest+news/', formatDateTime(utcNow(), 'yyyy/MM/dd'))
```



Expression builder: constructing the dated directory path for the sink.

A second expression builds a unique, timestamped **file name** for each run so results never collide or overwrite one another:

```
@concat('search-result_', formatDateTime(utcNow(), 'yyyymmdd_HH:MM:SS'), '.json')
```

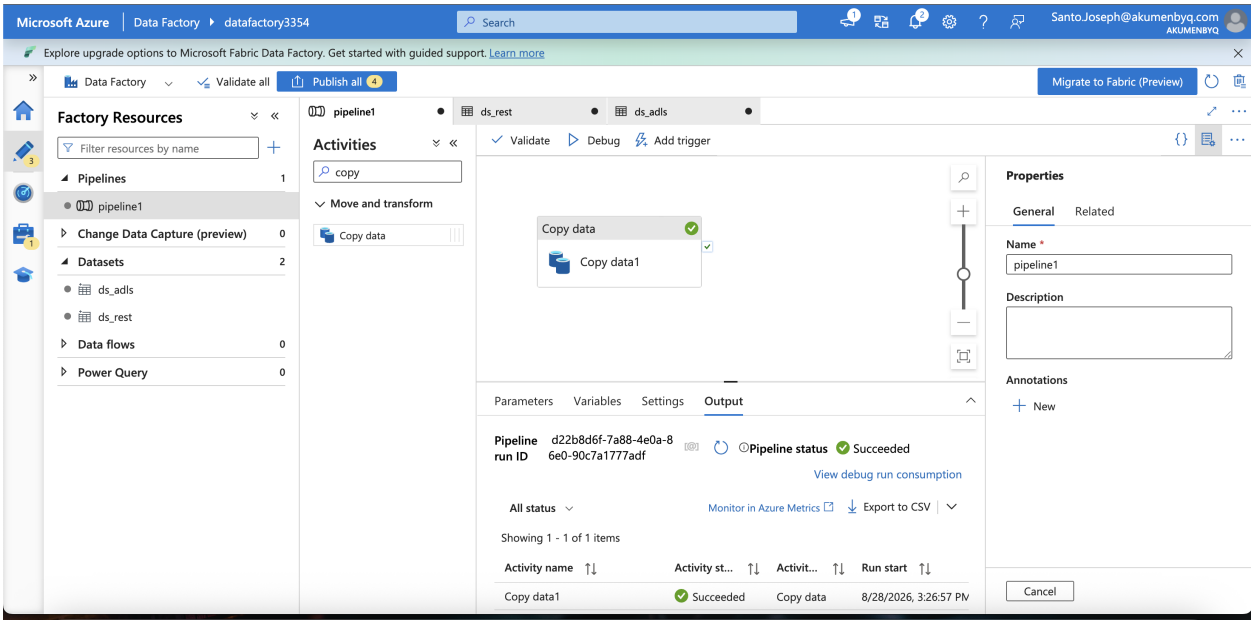


Expression builder: constructing the timestamped file name for the sink.

Together, the directory expression and file-name expression mean every pipeline run writes to a path like **output/searchapi/google/latest+news/2026/08/28/search-result_20260828_09-56-58.json**.

Step 9: Run the pipeline

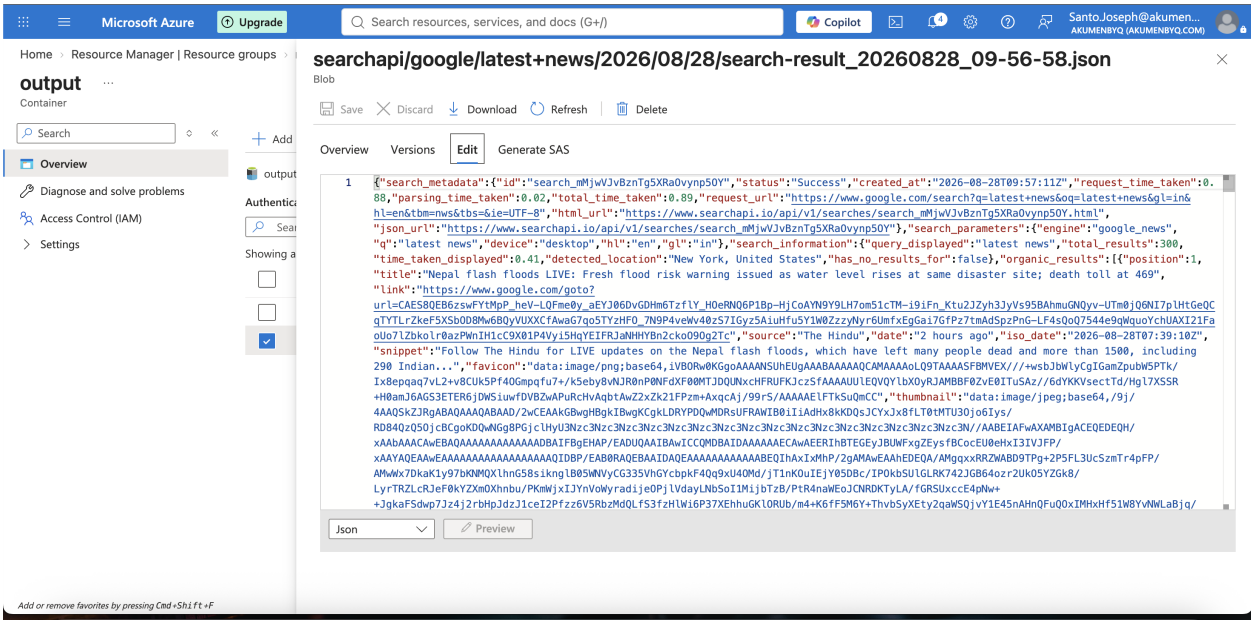
With **ds_rest** as the Copy Data source and **ds_adls** as the sink, running **Debug** on **pipeline1** succeeds. The Factory Resources panel shows both datasets (ds_adls, ds_rest) in place, and the run output confirms **Copy data1** succeeded.



Debug run succeeded, with ds_rest (source) and ds_adls (sink) both configured.

Step 10: Verify the dated JSON file landed in Blob Storage

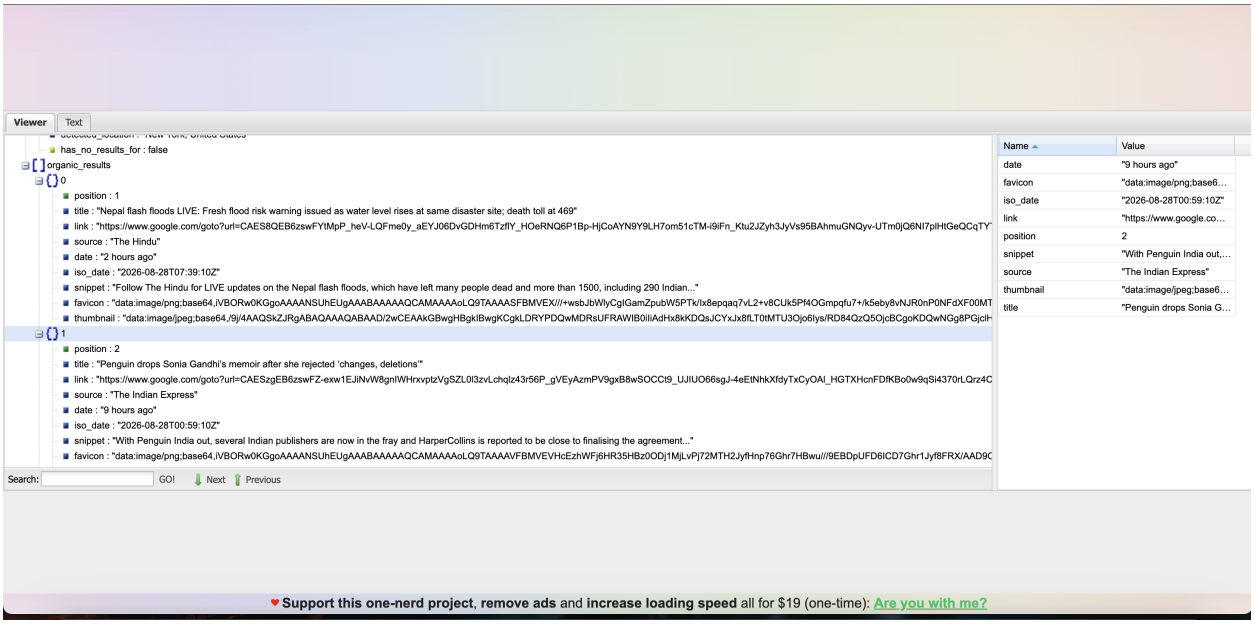
Browsing to the **output** container in the Storage account confirms the dynamic path worked as designed: the blob is named **search-result_20260828_09-56-58.json** and sits inside the **searchapi/google/latest+news/2026/08/28/** folder structure built in Step 8. Opening it shows the full Google News response — metadata, parameters, and the list of organic results.



The copied JSON blob, viewed in the Azure Portal at its dated, timestamped path.

Step 11: Inspect the results in a JSON viewer

Dropping the JSON into a viewer (jsonviewer.stack.hu) makes the article list easy to skim. Each entry under **organic_results** carries a position, title, link, source, a relative and ISO date, a snippet, and image data (favicon / thumbnail) — for example, live coverage of the Nepal flash floods from The Hindu, followed by stories from The Indian Express and Hindustan Times.



organic_results entries expanded in the JSON viewer: title, link, source, date, snippet.

Summary of the full flow

1. Pick the Google News endpoint and its required q parameter from the SearchApi.io docs.
2. Confirm the SearchApi account and copy the API key from the dashboard.
3. Make sure the target Data Factory and Storage account exist in the Azure resource group.
4. Create a REST linked service (ls_rest) in Data Factory pointed at the SearchApi base URL.
5. Build and note the full request URL (endpoint + query params + api_key).
6. Also create a Blob Storage linked service (ls_adls) as the eventual destination.
7. Create the ds_rest source dataset with the query string as its Relative URL, test the connection, and preview the live JSON response.
8. On the ds_adls sink dataset, use the pipeline expression builder to generate a dated folder path and a timestamped file name, so every run lands in its own file.
9. Run the Copy Data pipeline (source: ds_rest, sink: ds_adls) and confirm it succeeds.
10. Verify the resulting JSON file landed at the expected dated/timestamped path in Blob Storage.
11. Use a JSON viewer to explore the organic_results and confirm the article data came through correctly.